

VideoMind

A multimodal video question-answering system that retrieves both what was said and what was shown, then answers with evidence timestamped back to the moment in the video it came from.

Anit Kumar Maity MCA, Pondicherry University · Dept. of Computer Science Solo build · FastAPI / Streamlit / Pinecone / Groq

2 retrieval modalities fused per query — transcript text and video frames	384 / 512 embedding dimensions — BGE text index, CLIP ViT-B/32 visual index	0.6 / 0.4 text–visual fusion weighting, tuned before temporal reranking	34 automated unit + integration tests running in CI on every push
---	---	---	---

A transcript alone can't answer what a video shows

Ask a text-only chatbot to point to the moment a chart appears on screen, or which slide a claim was made against, and it has nothing to retrieve from — the audio track was never the whole video. Three kinds of questions expose the gap:

SPOKEN

"What did the speaker say about pricing?"
Answerable from the transcript alone — standard text retrieval over ASR output is enough.

VISUAL

"When does the diagram with three boxes appear?"
Never said aloud. Only recoverable by searching what the frames actually show, not what was spoken.

BOTH

"Where do they demo the feature they mentioned?"
Needs the two signals fused — the mention anchors the topic, the frames confirm the moment.

01 Ingest Upload or paste a YouTube URL. yt-dlp pulls the source, routed through a PO Token provider service for clients that now require one.	02 Transcribe & sample Audio is split into overlapping segments and transcribed concurrently (faster-whisper / Groq ASR); frames are sampled at a configurable rate.	03 Embed, dual-index Transcript chunks go to a 384-dim BGE text index; frames go to a 512-dim CLIP visual index — two separate Pinecone spaces, one video_id.	04 Retrieve & fuse Each question queries both indexes; scores combine at a 0.6 text / 0.4 visual weighting, then get reranked by timestamp proximity.	05 Answer & cite Groq LLM answers under a "cite only supplied evidence" system prompt; the response links back to the exact timestamp it came from.
---	---	--	--	--

What broke first, and what fixed it for good

Four issues surfaced during real testing, each traced to a specific root cause rather than patched around — the kind of debugging that only shows up once a pipeline runs against real audio, real load, and a free-tier API budget.

COLD START Warm the model before the frontend polls for it fastembed's BGE model downloaded lazily on first use, taking 30–40s — long enough to race Streamlit's status poller and crash the first request. Fixed with a FastAPI lifespan hook that pre-loads the model at startup, plus a guarded status check on the frontend.	RATE LIMITS Stay under Groq's TPM budget by design, not by luck Per-chunk summarization calls exhausted Groq's token-per-minute budget on longer videos. A sliding-window rate limiter now tracks spend and paces requests, instead of discovering the wall mid-run.
TIMEOUTS Give large uploads a timeout that matches reality Long-video FLAC uploads to Groq's ASR endpoint tripped httpx's default timeout. Fixed with explicit, configurable <code>httpx.Timeout</code> settings sized for actual file transfer time, not a generic default.	PORTABILITY Make storage and the LLM provider swappable, not hardcoded Local disk works in development but is wiped on every Render restart. A <code>StorageBackend</code> abstraction (local / S3) and an <code>LLMProvider</code> abstraction mean the deployment target changes via config, not code.

"A pipeline that only works on a warm cache and a fresh rate-limit window isn't finished — it's just untested against Monday morning."

— project design note

Built to run local-first, deploy cloud-ready

BACKEND

FastAPI

MongoDB Atlas

Pinecone

Groq

faster-whisper

CLIP ViT-B/32

bge-small-en-v1.5

bcrypt + PyJWT

FRONTEND & INFRA

Streamlit 1.38

Docker

GitHub Actions CI

AWS S3

yt-dlp + PO Token provider

Evaluation harness (evaluation/eval_runner.py) reports Recall@1/5/10, MRR, and temporal IoU against an MSR-VTT subset — built to benchmark retrieval quality as the dataset scales, independent of any single demo run.

Current state

Core pipeline	WORKING
Auth & per-user history	WORKING
CI (lint · type-check · tests)	GREEN
History retention	5 DAYS
YouTube ingestion	IN PROGRESS
Cloud deployment	IN PROGRESS